

Session 2 - Lab experiment

COURSE: DESIGN AND ANALYSIS OF ALGORITHMS

COURSE CODE: CSA0603

1.

```
n = int(input("Enter number of elements: "))
```

```
arr = list(map(int, input("Enter the elements: ").split()))
```

```
unique = []
```

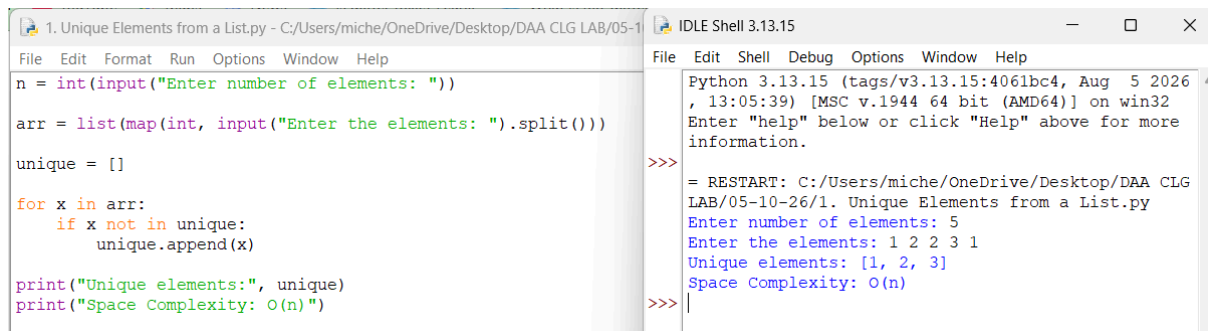
```
for x in arr:
```

```
    if x not in unique:
```

```
        unique.append(x)
```

```
print("Unique elements:", unique)
```

```
print("Space Complexity: O(n)")
```



```
1. Unique Elements from a List.py - C:/Users/miche/OneDrive/Desktop/DAA CLG LAB/05-10-26/1. Unique Elements from a List.py
File Edit Format Run Options Window Help
n = int(input("Enter number of elements: "))
arr = list(map(int, input("Enter the elements: ").split()))
unique = []
for x in arr:
    if x not in unique:
        unique.append(x)
print("Unique elements:", unique)
print("Space Complexity: O(n)")

Python 3.13.15 (tags/v3.13.15:4061bc4, Aug 5 2026, 13:05:39) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/miche/OneDrive/Desktop/DAA CLG LAB/05-10-26/1. Unique Elements from a List.py
Enter number of elements: 5
Enter the elements: 1 2 2 3 1
Unique elements: [1, 2, 3]
Space Complexity: O(n)
>>>
```

2.

```
n = int(input("Enter number of elements: "))
```

```
arr = list(map(int, input("Enter the elements: ").split()))
```

```
for i in range(n - 1):
```

```
    for j in range(n - 1 - i):
```

```
        if arr[j] > arr[j + 1]:
```

```
            arr[j], arr[j + 1] = arr[j + 1], arr[j]
```

```
print("Sorted array:", arr)
```

```
print("Time Complexity: O(n^2)")
```

```
Bubble Sort + Time Complexity.py - C:/Users/miche/OneDrive/Desktop/DAA CLG LAB/05-10-26/Bubble Sort + Time Complexity.py
File Edit Format Run Options Window Help
n = int(input("Enter number of elements: "))
arr = list(map(int, input("Enter the elements: ").split()))

for i in range(n - 1):
    for j in range(n - 1 - i):
        if arr[j] > arr[j + 1]:
            arr[j], arr[j + 1] = arr[j + 1], arr[j]

print("Sorted array:", arr)
print("Time Complexity: O(n^2)")

>>> = RESTART: C:/Users/miche/OneDrive/Desktop/DAA CLG LAB/05-10-26/Bubble Sort + Time Complexity.py
Enter number of elements: 5
Enter the elements: 64 34 25 12 22
Sorted array: [12, 22, 25, 34, 64]
Time Complexity: O(n^2)
>>> |
```

3.

```
n = int(input("Enter number of elements: "))
```

```
arr = list(map(int, input("Enter the sorted elements: ").split()))
```

```
x = int(input("Enter the number to search: "))
```

```
low = 0
```

```
high = n - 1
```

```
found = False
```

```
while low <= high:
```

```
    mid = (low + high) // 2
```

```
    if arr[mid] == x:
```

```
        found = True
```

```
        break
```

```
    elif arr[mid] < x:
```

```
        low = mid + 1
```

```
    else:
```

```
        high = mid - 1
```

```
if found:
```

```
    print("Element found")
```

```
else:
```

```
    print("Element not found")
```

```
print("Time Complexity: O(log n)")
```

The image shows a screenshot of a Python IDE with two windows. The left window, titled 'BINARY SEARCH.py', contains the following code:

```
n = int(input("Enter number of elements: "))
arr = list(map(int, input("Enter the sorted elements: ").split()))
x = int(input("Enter the number to search: "))

low = 0
high = n - 1
found = False

while low <= high:
    mid = (low + high) // 2

    if arr[mid] == x:
        found = True
        break
    elif arr[mid] < x:
        low = mid + 1
    else:
        high = mid - 1

if found:
    print("Element found")
else:
    print("Element not found")
print("Time Complexity: O(log n)")
```

The right window, titled 'IDLE Shell 3.13.15', shows the execution output:

```
= RESTART: C:/Users/miche/OneDrive/Desktop/DAA CLG LAB/05-10-26/BINARY SEARCH.py
Enter number of elements: 5
Enter the sorted elements: 10 20 30 40 50
Enter the number to search: 30
Element found
Time Complexity: O(log n)
```

4.

```
def merge_sort(arr):
```

```
    if len(arr) <= 1:
```

```
        return arr
```

```
    mid = len(arr) // 2
```

```
    left = merge_sort(arr[:mid])
```

```
    right = merge_sort(arr[mid:])
```

```
    result = []
```

```
    i = 0
```

```
    j = 0
```

```
    while i < len(left) and j < len(right):
```

```
        if left[i] <= right[j]:
```

```
            result.append(left[i])
```

```
            i += 1
```

```
        else:
```

```
            result.append(right[j])
```

```
            j += 1
```

```
    while i < len(left):
```

```
        result.append(left[i])
```

```
        i += 1
```

```
while j < len(right):
    result.append(right[j])
    j += 1
```

```
return result
```

```
n = int(input("Enter number of elements: "))
```

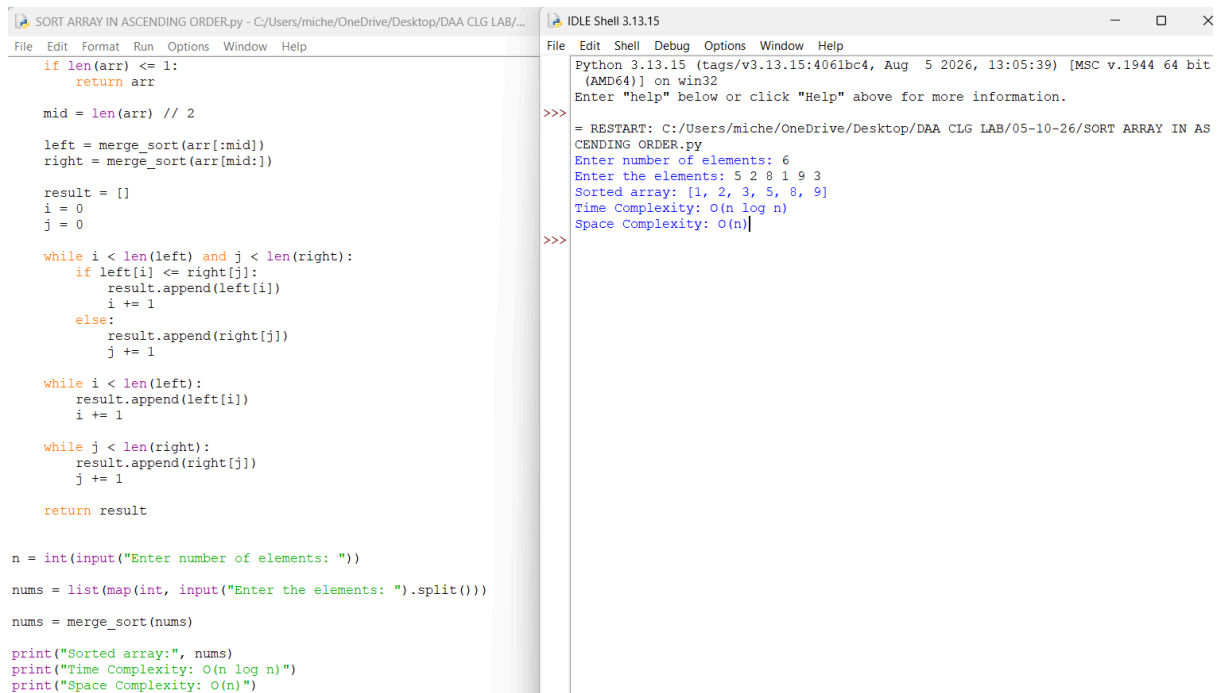
```
nums = list(map(int, input("Enter the elements: ").split()))
```

```
nums = merge_sort(nums)
```

```
print("Sorted array:", nums)
```

```
print("Time Complexity: O(n log n)")
```

```
print("Space Complexity: O(n)")
```



The screenshot displays a Python IDE with two windows. The left window, titled 'SORT ARRAY IN ASCENDING ORDER.py', contains the following code:

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    result = []
    i = 0
    j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    while i < len(left):
        result.append(left[i])
        i += 1

    while j < len(right):
        result.append(right[j])
        j += 1

    return result

n = int(input("Enter number of elements: "))
nums = list(map(int, input("Enter the elements: ").split()))
nums = merge_sort(nums)

print("Sorted array:", nums)
print("Time Complexity: O(n log n)")
print("Space Complexity: O(n)")
```

The right window, titled 'IDLE Shell 3.13.15', shows the execution output:

```
Python 3.13.15 (tags/v3.13.15:4061bc4, Aug 5 2026, 13:05:39) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> = RESTART: C:/Users/miche/OneDrive/Desktop/DAA CLG LAB/05-10-26/SORT ARRAY IN ASCENDING ORDER.py
Enter number of elements: 6
Enter the elements: 5 2 8 1 9 3
Sorted array: [1, 2, 3, 5, 8, 9]
Time Complexity: O(n log n)
Space Complexity: O(n)
>>>
```